

Till now whatever we have studied
are called foundational Design
Pattern

↓ used in
most of the problem or Generic
problem

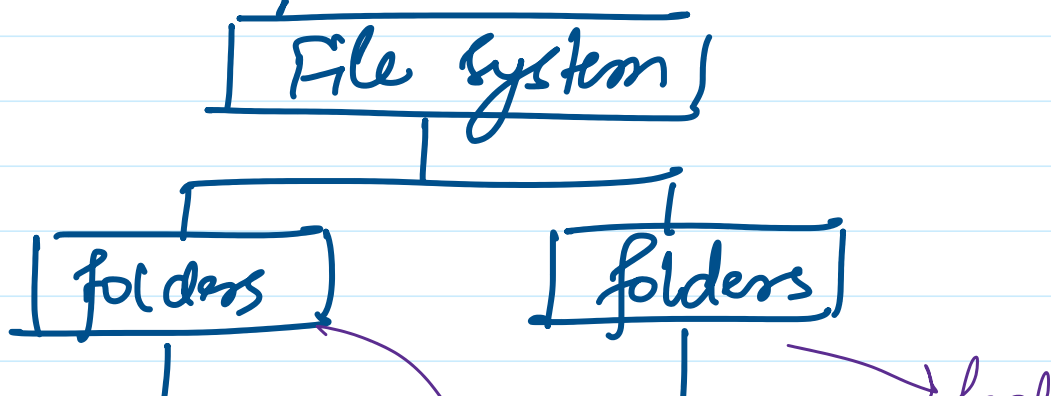
from now whatever we will study
will NOT be used for Generic
purpose. They will be used for
some specific use-cases/problems.

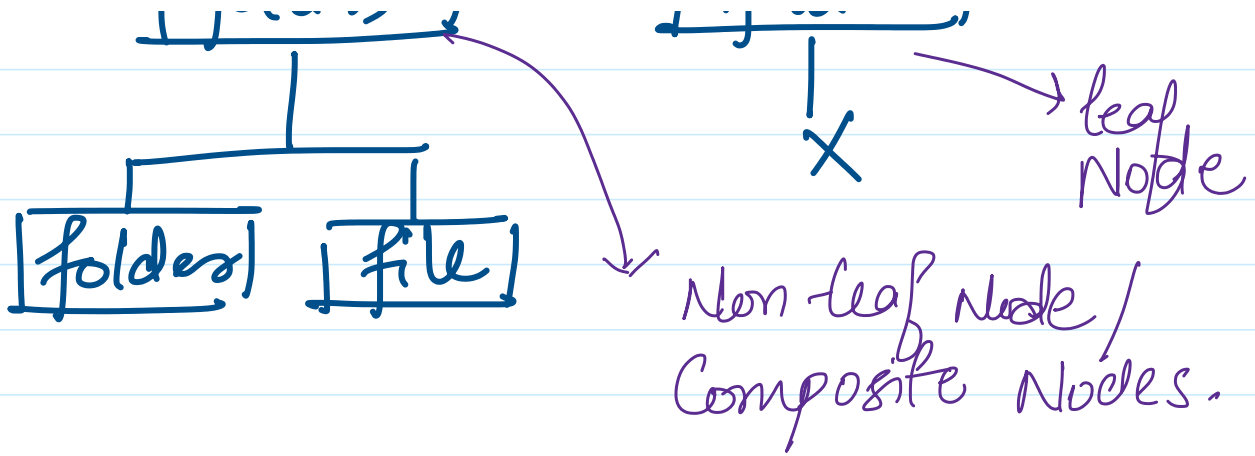
Composite Design Pattern.

any problem which you express in
form of trees / hierarchical form
can be solved by
Composite Design Pattern

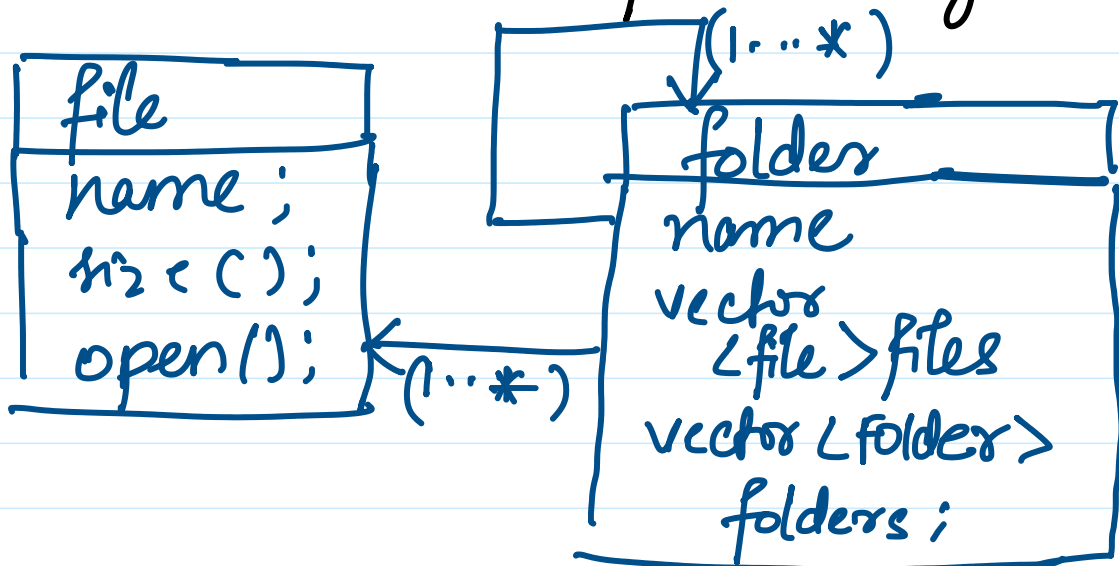
Example

folder system.





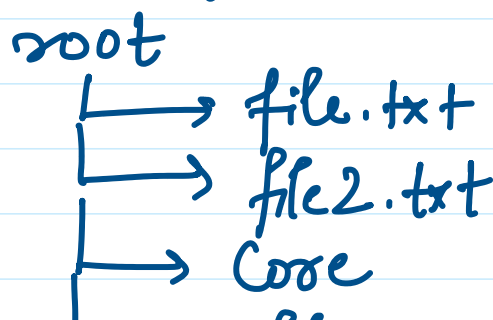
How we will design file system if we don't know Composite Design Pattern.

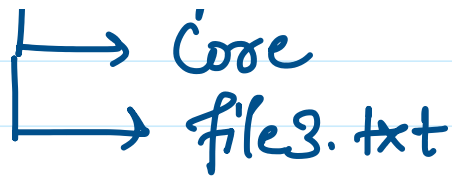


Now if we want implement

1. ls — command

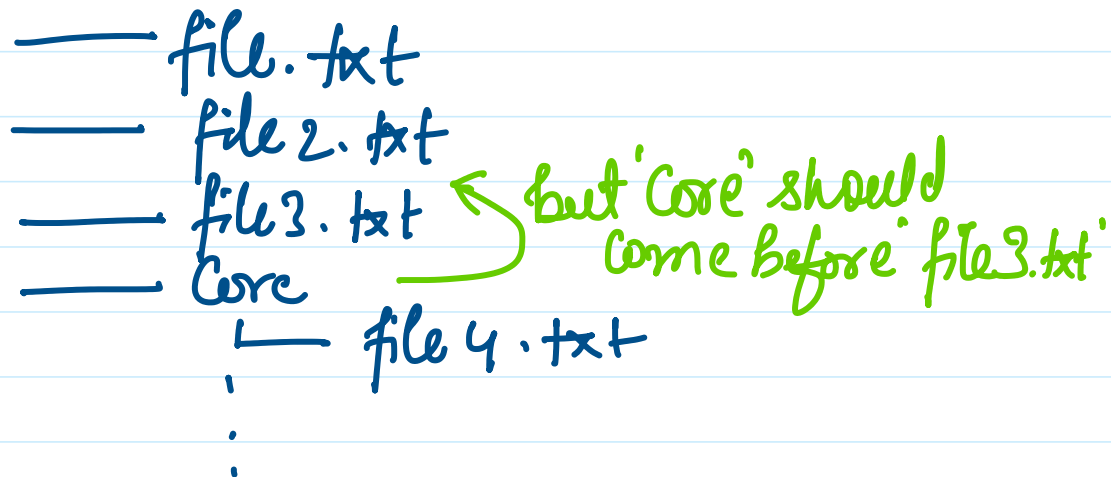
then it will loop through 'files' & 'folders' one after another and print it





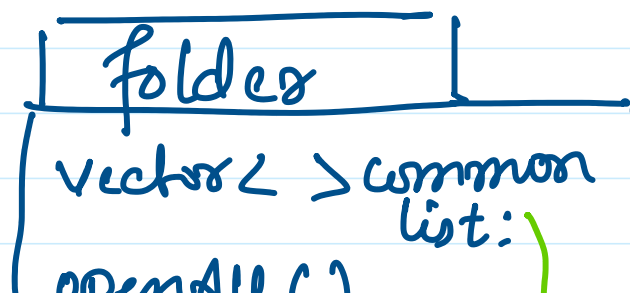
2. `OpenAll()` - this command will open all hierarchy recursively

But here order will be not correct b/c it first loop through all files then through all folders and order will look like this



Same for 'ls' command also - order will not be maintained

Now to resolve this issue we must have a common list so that order is maintained



list:
 openAll()

Now to implement openAll() { }
 our code will be:

```
for (element in list) {
  if (element == file) {
    cout << element
      << "\n";
  }
  else {
    element->openAll();
  }
}
```

↳ we are calling
 Recursively

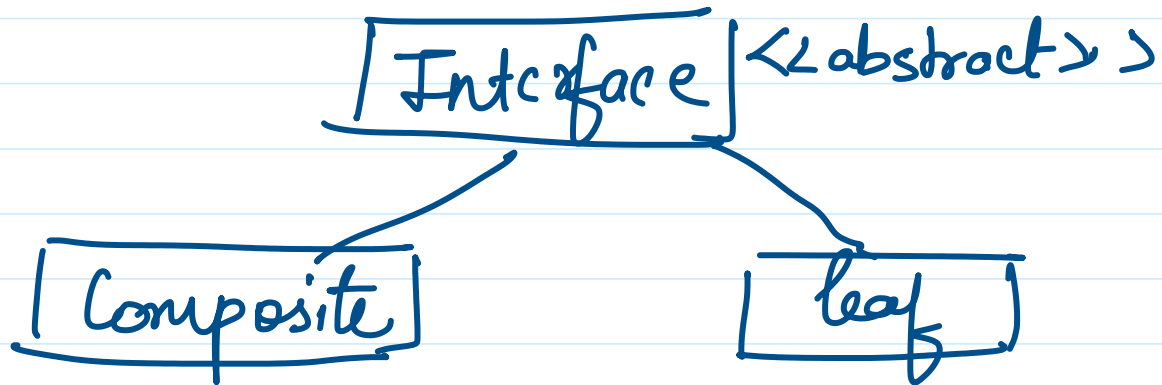
So, we are complicating our things.
 Suppose in future more command
 comes like - 'cd'

↳ to Change Directory

∴ Our Implementation is getting
 more complex.

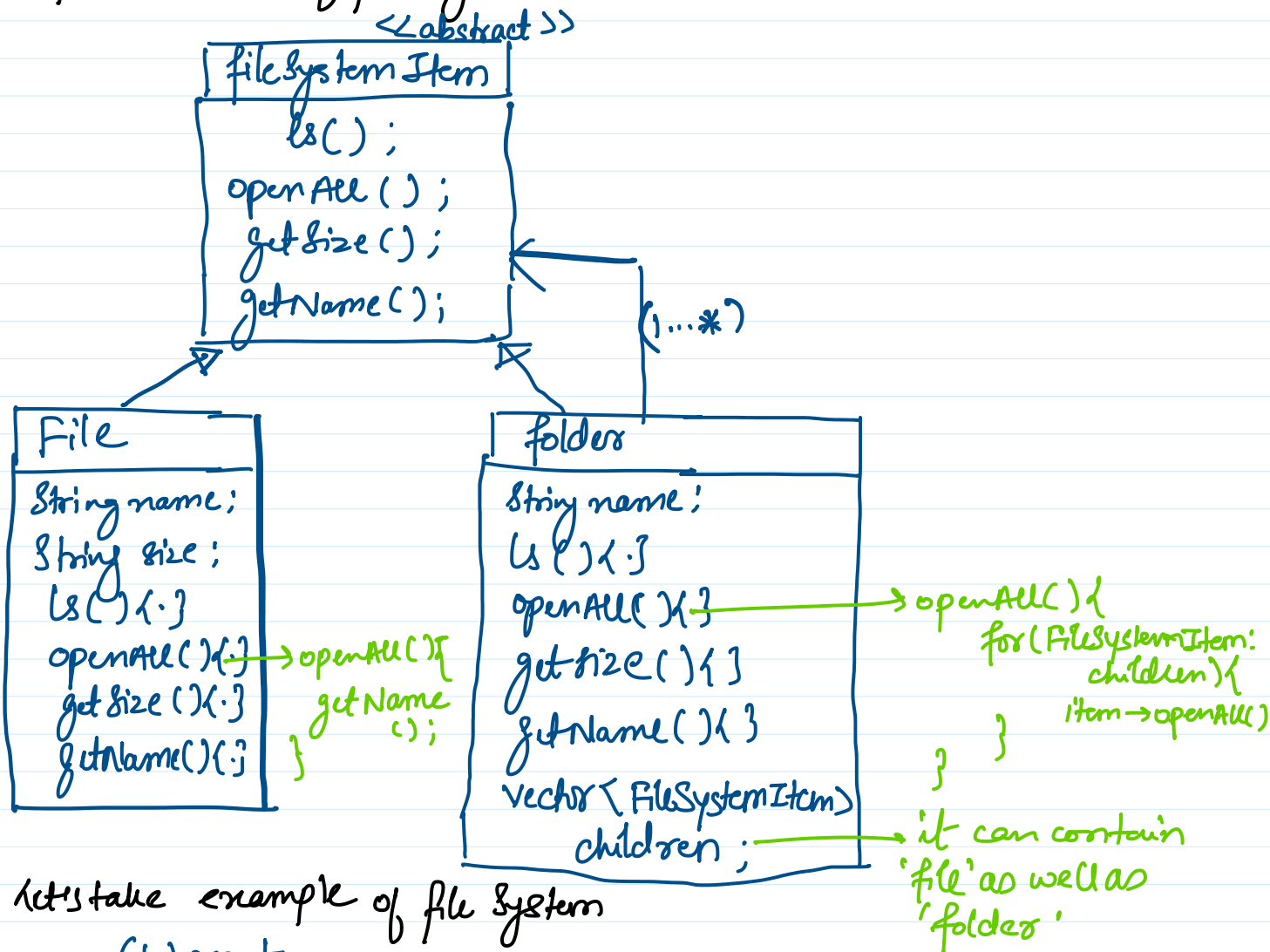
So Composite Design Pattern says,
 You have two elements
 - Composite & Leaf and

You have two elements
- Composite & leaf and
treat them same together
means they must have same interface
and they only implement same method
declared in Interface.



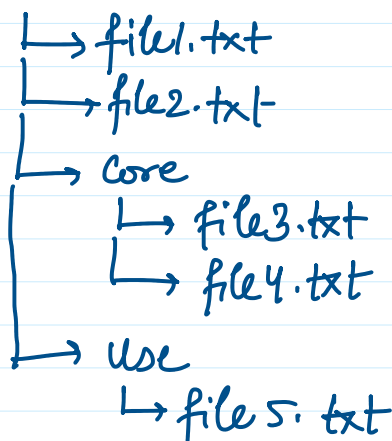
By doing this we don't have to
do if, else.

Implementation of file system



Let's take example of file system

(+) root



Now suppose we call `openAll()` at root. Now root don't know whether it is a 'file' or 'folder' so it will simply call `openAll()`

simply call openAll()

↓
Dynamic Polymorphism.

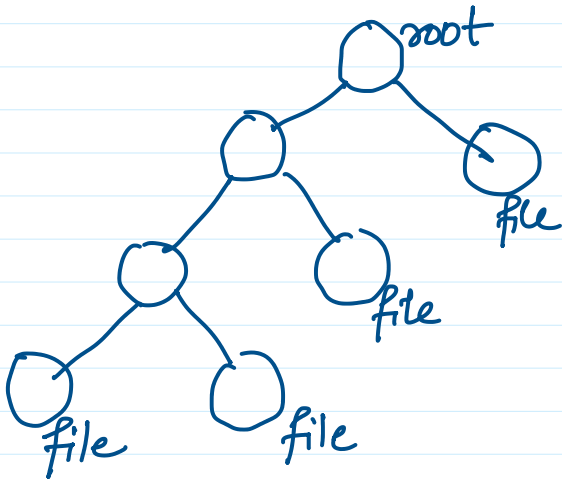
basic principle we know

```
for (FileSystemItem item:  
      children) {
```

```
    item → openAll();
```

→ without
thinking what
the 'item' is
it simply
call
openAll()

So, there is a Recursion like we do
in Trees Data Structure.



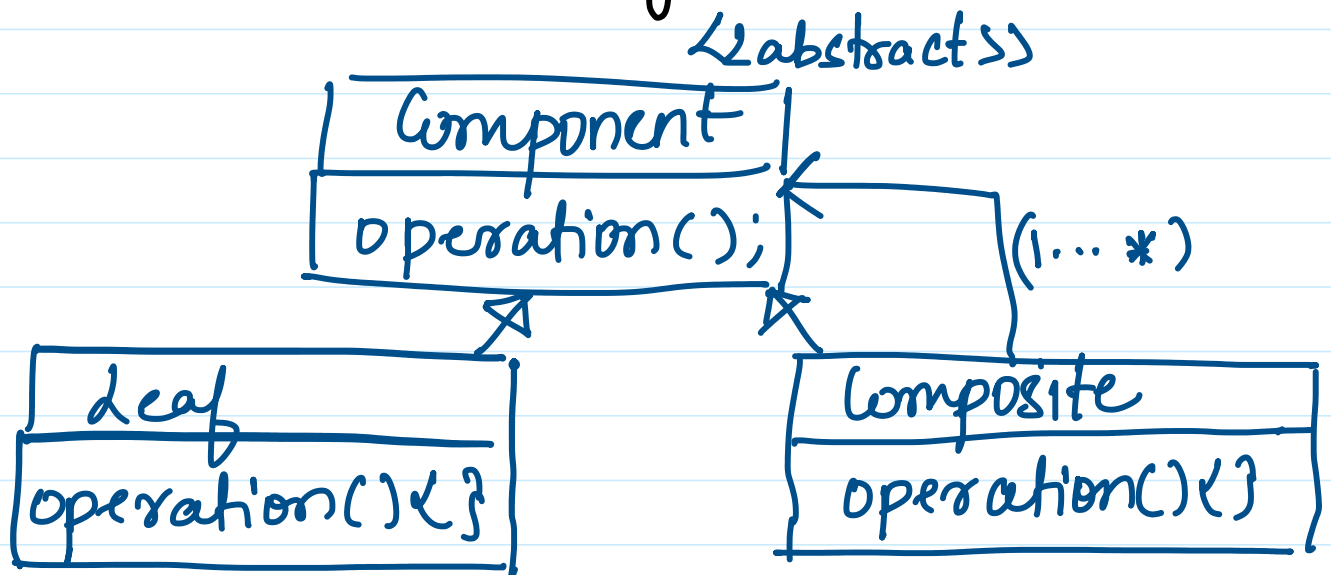
∴ In Composite Design Pattern we
treat Composite key & leaf key
same

So, we got a benefit to implement
same method in different ways

So, root doesn't know it has a folder
or file it just need to call openAll()

So, if-else Statement NOT required
& ordering also solved.

Standard UML Diagram



Code → see Notes...

Standard Definition:-

Composite Pattern composes object into tree like structure representing a part-whole hierarchy. It let client treats individually object and compositions of object uniformly.

Use Cases

1. Drop down System
2. file system

2. file system